Yashaswi's Education Society's
International Institute of Management Studies

Introduction to Operating System:

Mr. Prashant Wadkar

Prerequisite:

- Basics of Operating System

Objectives

- 1. To learn the fundamentals of Operating Systems and handle processes and threads and their communication
- 2. To learn the mechanisms involved in memory management in contemporary OS
- 3. To know the functionality of Multiprocessor OS and Mobile OS.
- 4. To gain knowledge on distributed operating system concepts.
- 5. To learn about Basics of Linux.
- 6. To learn programmatically to implement Linux OS mechanisms.

Course Outcomes

- : Student will be able to
- CO1: Understand structure of OS, process management and synchronization. (Understand)
- CO2: Understand multicore and multiprocessing OS. (Understand)
- CO3: explain Realtime and embedded OS (Understand)
- CO4: understand Windows and Linux OS fundamentals and administration. (Understand)
CO5: solve shell scripting problems (Apply)

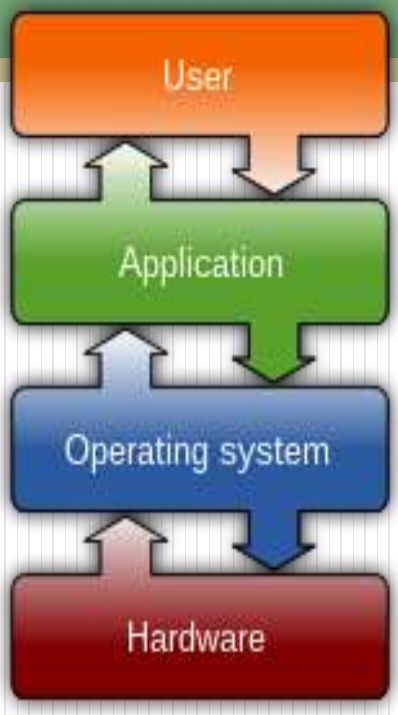
Chapter 1: Introduction

- 1. Overview
- 1.1. Overview of operating systems
- 1.2 Functionalities and Characteristics of OS
- 1.3 Hardware concepts related to OS
- 1.4 CPU states
- 1.5 I/O channels
- 1.6 Memory Management
- 1.6.1 Memory Management Techniques
- 1.6.2 Contiguous & Non-Contiguous allocation
- 1.6.3 Logical & Physical Memory Conversion of Logical to Physical address

Chapter 1: Introduction

- **1.7 Paging**
- **1.7.1 Demand Paging**
- **1.7.2 Page Replacement Concept**
- **1.8 Segmentation - Segment with paging**
- **1.9 Virtual Memory Concept**
- **1.10 Thrashing**
- **Extra Reading: Type of OS, Batch OS, Time Sharing OS, Network OS, Multiprogramming OS, Multiprocessing OS, Evolution of Operating System., Computer System Organization Operating System Structure and Operations- System Calls, System Programs, OS Generation and System Boot**

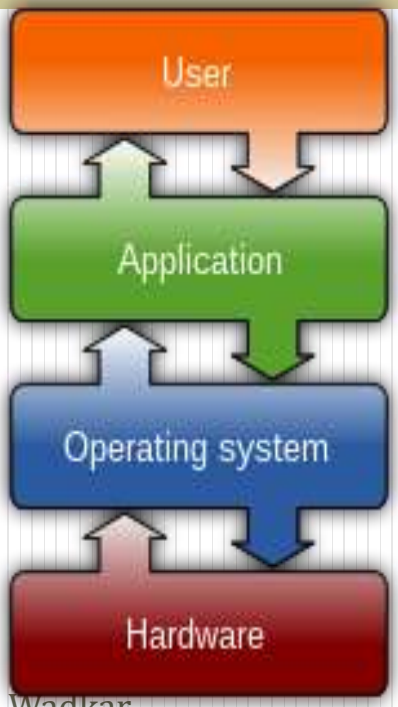
"What is Operating System???"



An *operating system acts as an intermediary between the user of a* computer and the computer hardware. The purpose of an operating system is to provide an environment in which a user can execute programs in a *convenient and efficient manner.*

An operating system is software that manages the computer hardware.

"What is Operating System???"



Common features
•Process management •Interrupts •Memory management •File system •Device drivers •Networking •Security •I/O

What is Operating System???”

- An **operating system** (**OS**) is system software that manages computer hardware, software resources, and provides common services for computer programs.
- E.g. Sun Microsystems' Solaris; Linux; Mach; Microsoft MS-DOS, Windows NT, Windows 2000, and Windows XP; DEC VMS and TOP5-20; IBM OS/2; and Apple Mac OS X.

- **System software** is software designed to provide a platform for other software. Examples of system software include operating systems (OS) like macOS, Linux, Android and Microsoft Windows, computational science software, game engines, search engines, industrial automation, and software as a service applications.[1]

1. Overview

- An operating system is a program that manages the computer hardware. It also provides a basis for application programs and acts as an intermediary between the computer user and the computer hardware.

- **Mainframe operating systems** are designed primarily to optimize utilization of hardware. **Personal computer (PC) operating systems** support complex games, business applications, and everything in between. Operating systems for handheld computers are designed to **provide an environment** in which a user can easily interface with the computer to execute programs. Thus, some operating systems are designed to be ***convenient***, *others to be* ***efficient,*** *and others* some **combination of the two.**

1. Overview

- A computer system can be divided roughly into four components: the *hardware, the operating system, the application programs,* and the *users (Figure 1.1).*

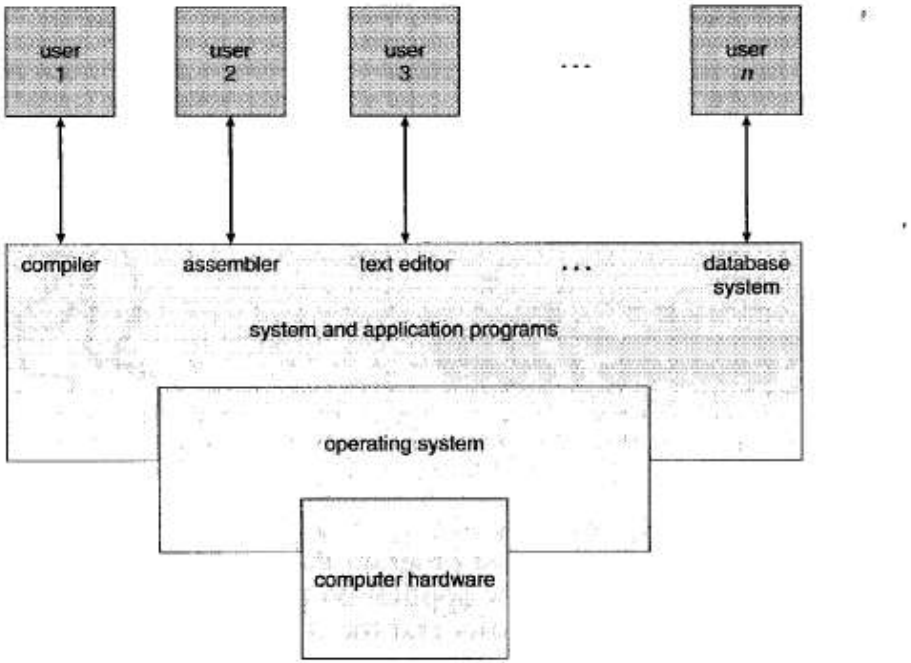


Figure 1.1 Abstract view of the components of a computer system.

1. Overview

- The **hardware**-the central processing unit (CPU), the memory, and the input/output (VO) devices-provides the basic computing resources for the system.

- The **application programs**-such as word processors, spreadsheets, compilers, and web browsers-define the ways in which these resources are used to solve users' computing problems.

- The operating system **controls and coordinates the use of the hardware among the various application programs** for the various users. We can also **view** a computer system as consisting of hardware, software, and data.

- The operating system provides the means for proper use of these resources in the operation of the computer system. An operating system is **similar to a *government*.** *Like a government, it performs no useful function by* itself. **It simply provides an *environment within which other programs can do* useful work.**

1. Overview

- Most computer users sit in front of a **PC**, consisting of a monitor, keyboard, mouse, and system unit. Such a system is designed for **one user to monopolize its resources.** The goal is to maximize the work (or play) that the user is performing. In this case, the **operating system is designed mostly for ease of use, with some attention paid to performance and none paid to resource utilization-how various hardware and software resources are shared.** Performance is, of course, important to the user; but rather than **resource utilization**, such systems are **optimized for the single-user experience.**

- In other cases, a user sits at a terminal connected to a **mainframe or minicomputer.** Other users are accessing the same computer through other terminals. These users **share resources** and may **exchange information.** The operating system in such cases is designed **to maximize resource utilizationton** assure that all available CPU time, memory, and I/O are used efficiently

1. Overview

- In still other cases, users sit at **workstations** connected to networks of other workstations and servers. **These users have dedicated resources at their disposal, but they also share resources such as networking and servers-file, compute, and print servers**. Therefore, their operating system is designed to compromise between individual usability and resource utilization.

- Recently, many varieties of **handheld computers** have come into fashion. Most of these devices are standalone units for individual users. Some are connected to networks, either directly by wire or (more often) through wireless modems and networking. Because of power, speed, and interface limitations, they perform relatively few remote operations. Their operating systems are designed mostly for individual usability.

- Some computers have little or no user view. For example, **embedded computers** in **home devices and automobiles may have numeric keypads and may turn indicator lights on or off to show status**, but they and their operating systems are **designed primarily to run without user intervention**.

1. Overview

- In still other cases, users sit at **workstations** connected to networks of other workstations and servers. **These users have dedicated resources at their disposal, but they also share resources such as networking and servers-file, compute, and print servers**. Therefore, their operating system is designed to compromise between individual usability and resource utilization.

- Recently, many varieties of **handheld computers** have come into fashion. Most of these devices are standalone units for individual users. Some are connected to networks, either directly by wire or (more often) through wireless modems and networking. Because of power, speed, and interface limitations, they perform relatively few remote operations. Their operating systems are designed mostly for individual usability.

- Some computers have little or no user view. For example, **embedded computers** in **home devices and automobiles may have numeric keypads and may turn indicator lights on or off to show status**, but they and their operating systems are **designed primarily to run without user intervention**.

1. Overview

- start running- when it is powered up or rebooted-it needs to have an initial program to run.
- This **initial program, or bootstrap program**, tends to be simple. Typically, it is stored in **read-only memory (ROM)** or electrically erasable programmable read-only memory (EEPROM), known by the general term firmware, within the computer hardware. It initializes all aspects of the system, from CPU registers to device controllers to memory contents. The **bootstrap program** must know how to load the operating system and to start executing that system.
- To accomplish this goal, the bootstrap program must locate and load into memory the operating system kernel. The operating system then starts executing the first process, such as **"init,"** and waits for some event to occur.

- The occurrence of an event is usually **signaled by an interrupt** from either
- the **hardware or the software**. Hardware may trigger an interrupt at any time
- by sending a **signal to the CPU**, usually by way of the system bus. Software
- may trigger an interrupt by executing a special operation called a **system call**
- **(also called a monitor call).**

1. Overview

- **Interrupt**

- **1. Hardware -** by sending a **signal to the CPU**,
- **2. Software**. -may trigger an interrupt by executing a special operation called a **system call (monitor call).**

- Interrupts are an important part of a computer architecture. Each computer design has its own interrupt mechanism, but several functions are common. The interrupt must transfer control to the appropriate interrupt service routine.

1. Overview

- **Interrupt**

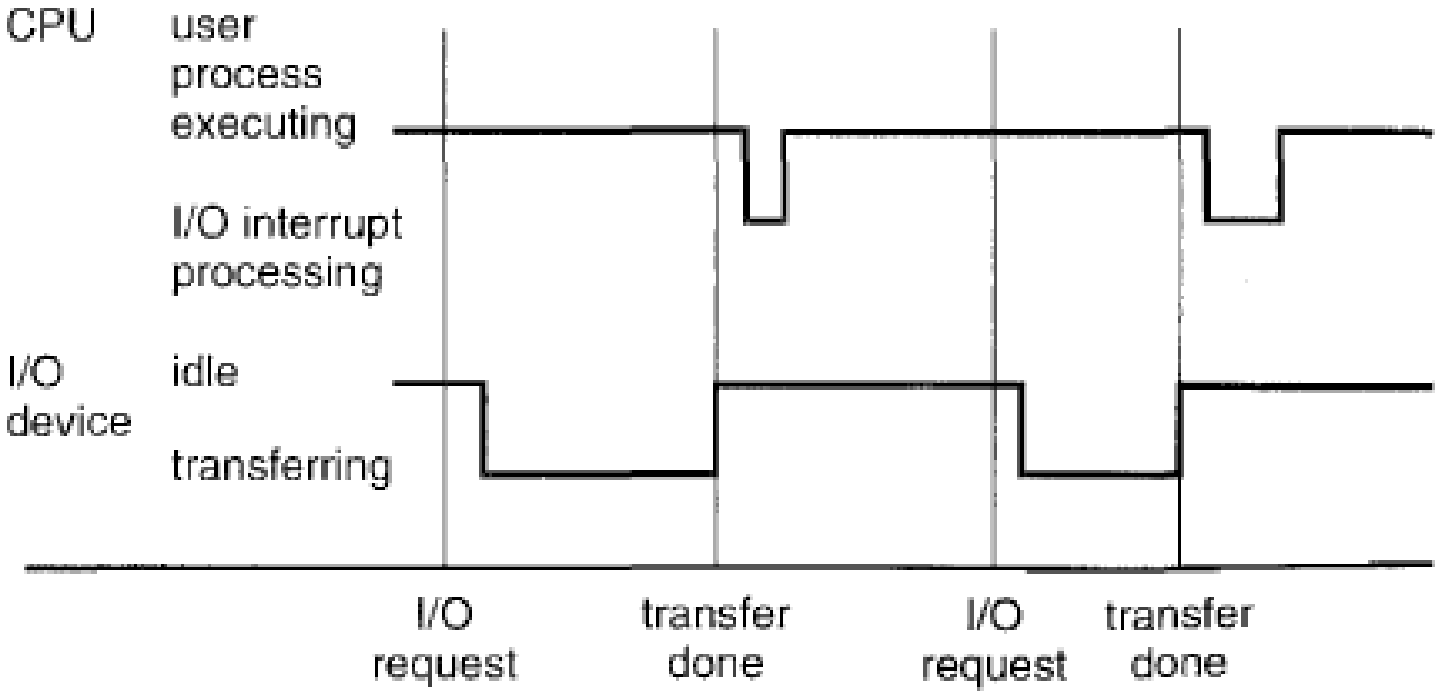


Figure 1.3 Interrupt time line for a single process doing output.

- **Environment might be**
  - **Multiple processes (Multiple task parallel)**
  - **Multiple CPU's- (But normally 1 CPU(how it is possible in execute multiple processes?)**
  - **Solution might be to go for multiple processes execution or go for Threading**
  - ***One type of process executing multiple times-e.g. Print server**
  - **(same process of printing by multiple users)**
- whats can do.?
  - 1. Create multiple Child processes or
  - 2. Perform multithreading environment
  - E.g.

1. Overview

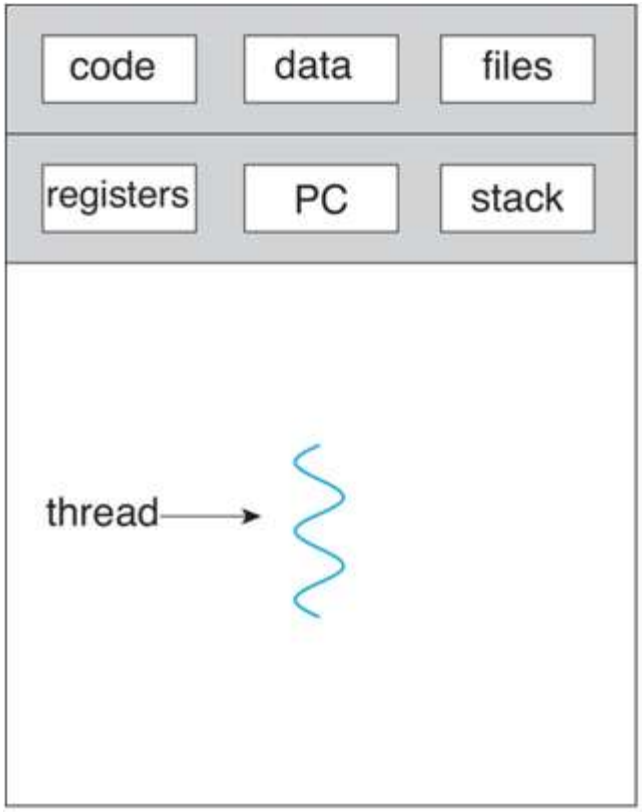
- **Interrupt**

- Method is to invoke a generic routine to examine the interrupt information; the routine, in turn, would call the interrupt-specific handler.
- However, interrupts must be handled quickly. Since only a predefined number of interrupts is possible table of pointers to interrupt routines can be used instead to provide the necessary
- speed. The interrupt routine is called indirectly through the table, with no intermediate routine needed.

1. Overview

- **Interrupt**
- Generally, the **table of pointers is stored in low memory** (the first 100 or so locations).
- These locations **hold the addresses of the interrupt service routines** for the various devices. This array, or interrupt vector, of addresses is then indexed by a **unique device number** t given with the interrupt request, to provide the address of the interrupt service routine for the interrupting device.

Single Processe

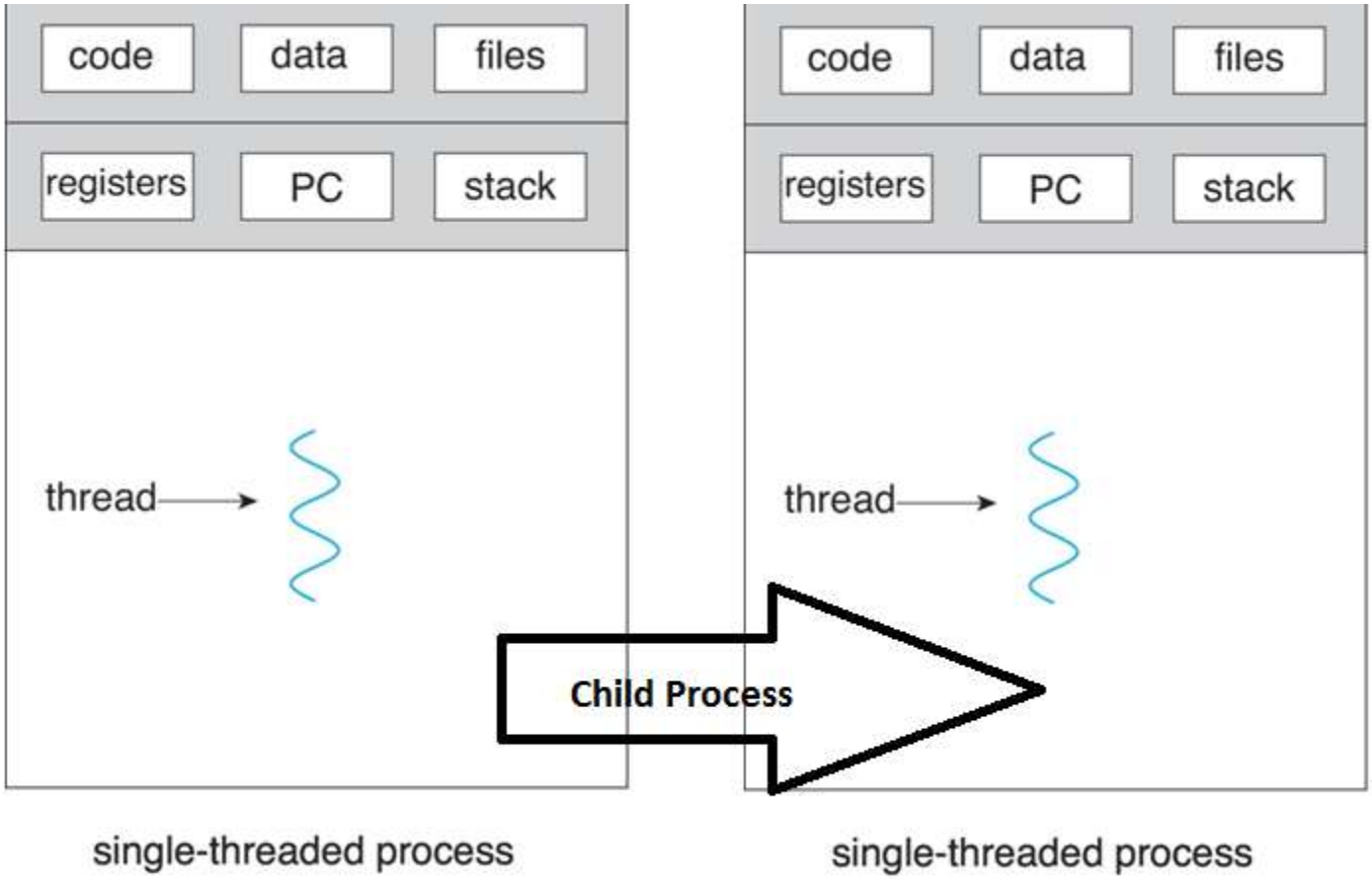


single-threaded process

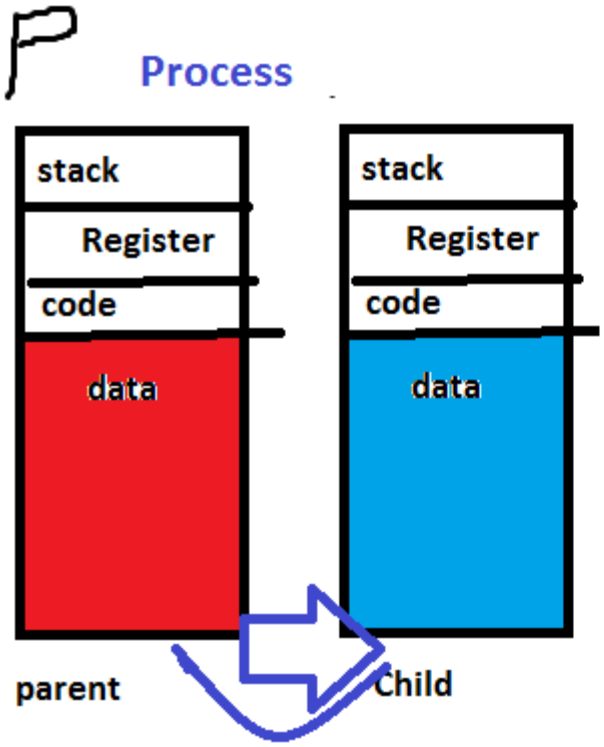
Process & Threads

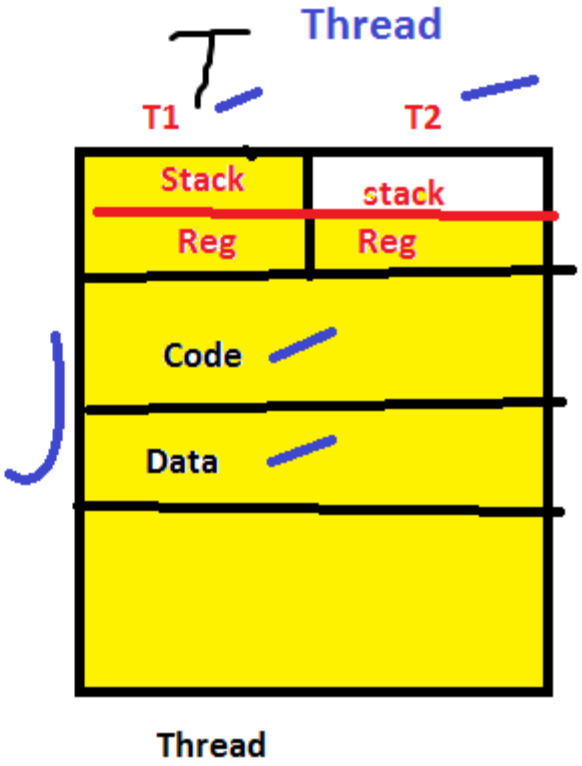
- **Environment might be**
  - **Multiple processes (Multiple task parallel)**
  - **Multiple CPU's- (But normally 1 CPU(how it is possible in execute multiple processes?)**
  - **Solution might be to go for multiple processes execution or go for Threading**
  - ***One type of process executing multiple times-e.g. Print server**
  - **(same process of printing by multiple users)**
- whats can do.?
  - 1. Create multiple Child processes or
  - 2. Perform multithreading environment
  - E.g.

Process & Child Process

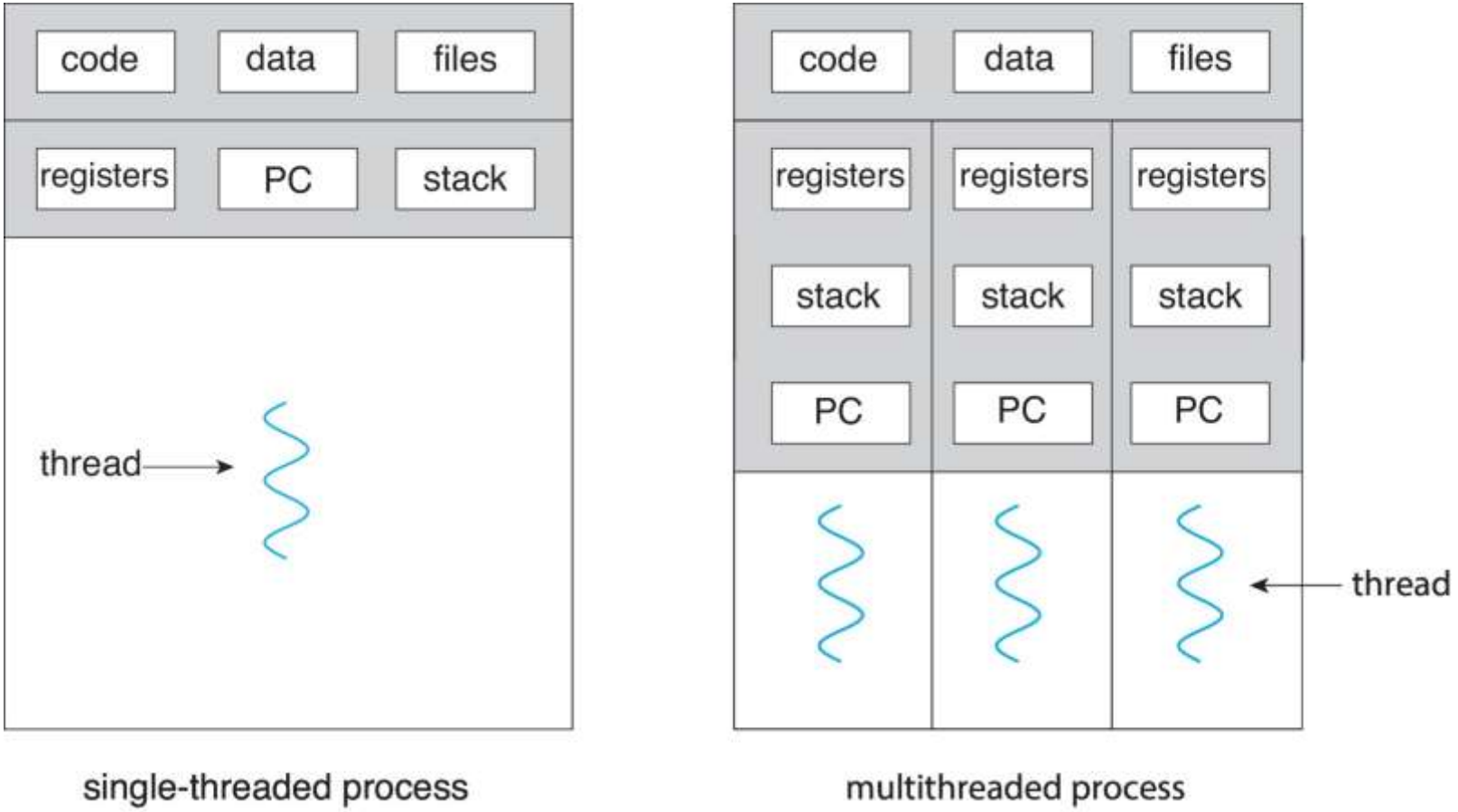


Process & Child Process





Single and Multithreaded Processes



single-threaded process

multithreaded process

Process & Threads

In Mutlitasking Environment	
Processes	**Threads**
0. Heavy weight task	0. Light weight task
1. System calls involved in process(fork())	1. There is no system call involved
2. OS treats different process differently	2. All user level threads treated as single level taks for OS
3. Different processes have different copyies of data, files, code	3. Thread share same copy of code and data
4. Context switching (time slicing) is slower (i.e Process Control Block to OS)	4. Context switching (time slicing) is faster (i.e Process Control Block to OS)
5. blocking a process will not block another process	5. Blocking a thread will block entire process
6. Independent	6. Interdependent

What is Memory Management?

- **Memory Management** is the process of controlling and coordinating computer memory, assigning portions known as blocks to various running programs to optimize the overall performance of the system.
- It is the most important function of an operating system that manages primary memory. It helps processes to move back and forward between the main memory and execution disk. It helps OS to keep track of every memory location, irrespective of whether it is allocated to some process or it remains free.

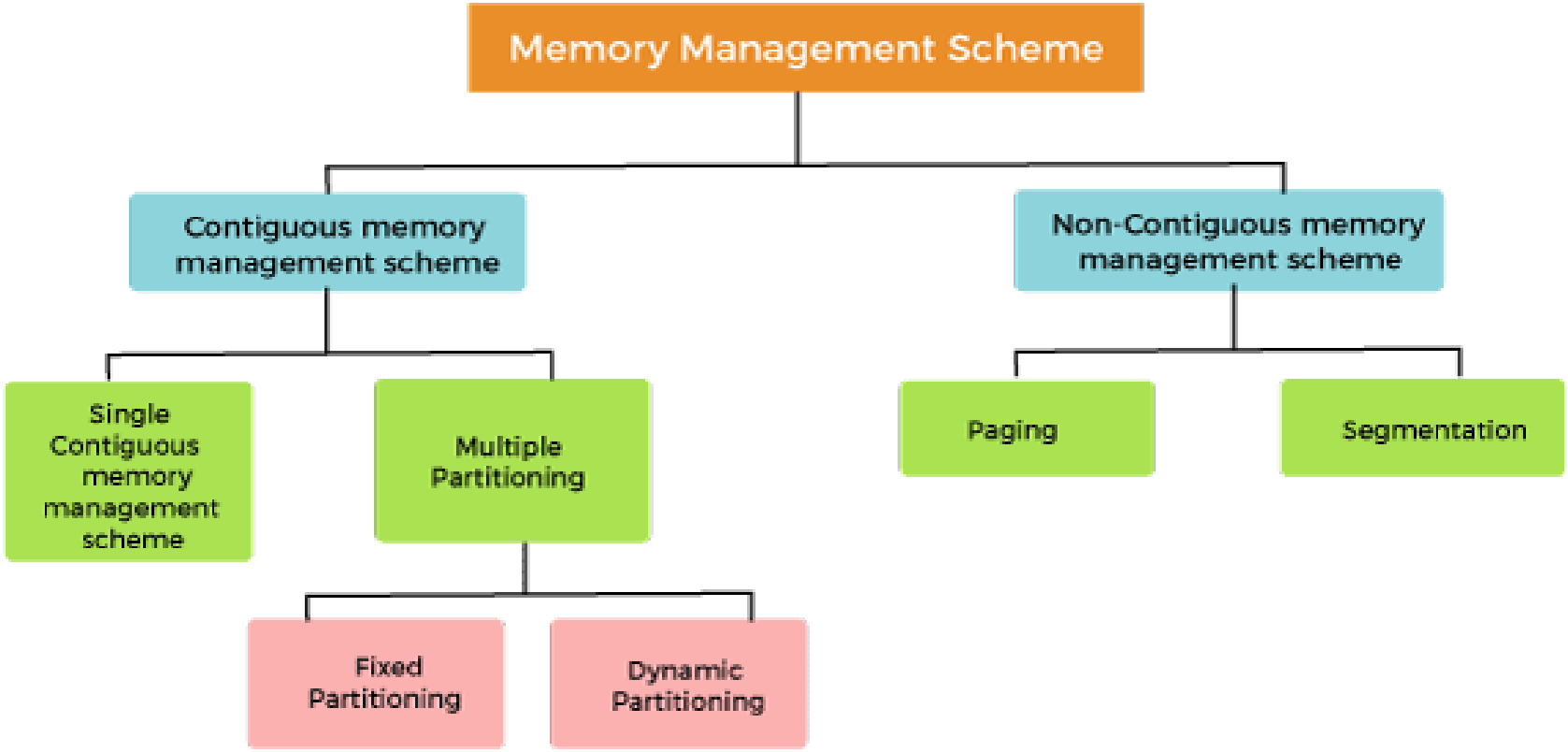
What is Memory Management?

- What is Memory Management?
- Why Use Memory Management?
- Memory Management Techniques
- What is Swapping?
- What is Memory allocation?
- What is Paging?
- What is Fragmentation Method?
- What is Segmentation?
- What is Dynamic Loading?
- What is Dynamic Linking?
- Difference Between Static and Dynamic Loading
- Difference Between Static and Dynamic Linking

Why Use Memory Management?

-
- Here, are reasons for using memory management:
- It allows you to check how much memory needs to be allocated to processes that decide which processor should get memory at what time.
- Tracks whenever inventory gets freed or unallocated. According to it will update the status.
- It allocates the space to application routines.
- It also make sure that these applications do not interfere with each other.
- Helps protect different processes from each other
- It places the programs in memory so that memory is utilized to its full extent.

Memory Management Techniques/Schemes



Classification of memory management schemes

Memory Management Techniques

- **Single Contiguous Allocation**
- It is the easiest memory management technique. In this method, all types of computer's memory except a small portion which is reserved for the OS is available for one application. For example, MS-DOS operating system allocates memory in this way. An embedded system also runs on a single application.
- **Partitioned Allocation**
- It divides primary memory into various memory partitions, which is mostly contiguous areas of memory. Every partition stores all the information for a specific task or job. This method consists of allotting a partition to a job when it starts & unallocated when it ends.
- **Paged Memory Management**
- This method divides the computer's main memory into fixed-size units known as page frames. This hardware memory management unit maps pages into frames which should be allocated on a page basis.
- **Segmented Memory Management**
- Segmented memory is the only memory management method that does not provide the user's program with a linear and contiguous address space.
- Segments need hardware support in the form of a segment table. It contains the physical address of the section in memory, size, and other data like access protection bits and status.

Memory Management Techniques

- **Contiguous memory management schemes:**
- In a Contiguous memory management scheme, each program occupies a single contiguous block of storage locations, i.e., a set of memory locations with consecutive addresses.
- **Non-Contiguous memory management schemes:**
- In a Non-Contiguous memory management scheme, the program is divided into different blocks and loaded at different portions of the memory that need not necessarily be adjacent to one another. This scheme can be classified depending upon the size of blocks and whether the blocks reside in the main memory or not.

Contiguous & non-contiguous memory management schemes

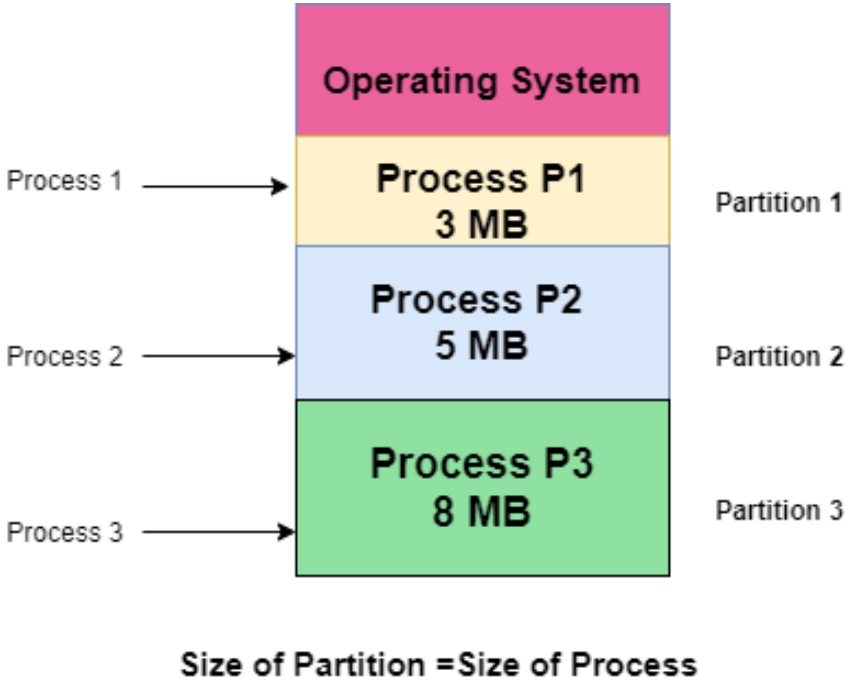


Fig 1 Contiguous memory management schemes

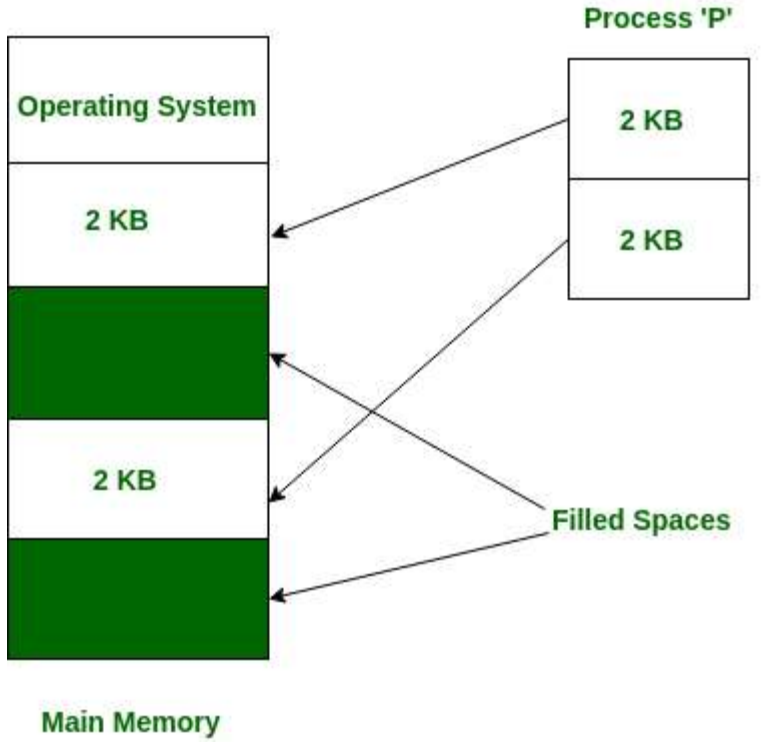


Fig. 2 non-contiguous memory management schemes

Memory Management Techniques

- **Single contiguous memory management schemes:**
- The Single contiguous memory management scheme is the simplest memory management scheme used in the earliest generation of computer systems. In this scheme, the main memory is divided into two contiguous areas or partitions. The operating systems reside permanently in one partition, generally at the lower memory, and the user process is loaded into the other partition.
- **Advantages of Single contiguous memory management schemes:**
- Simple to implement.
- Easy to manage and design.
- In a Single contiguous memory management scheme, once a process is loaded, it is given full processor's time, and no other processor will interrupt it.
- **Disadvantages of Single contiguous memory management schemes:**
- Wastage of memory space due to unused memory as the process is unlikely to use all the available memory space.
- The CPU remains idle, waiting for the disk to load the binary image into the main memory.
- It can not be executed if the program is too large to fit the entire available main memory space.
- It does not support multiprogramming, i.e., it cannot handle multiple programs simultaneously.

Memory Management Techniques

- **Multiple Partitioning:**
- The single Contiguous memory management scheme is inefficient as it limits computers to execute only one program at a time resulting in wastage in memory space and CPU time. The problem of inefficient CPU use can be overcome using multiprogramming that allows more than one program to run concurrently. **To switch between two processes, the operating systems need to load both processes into the main memory.** The operating system needs to divide the available main memory into multiple parts to load multiple processes into the main memory. Thus multiple processes can reside in the main memory simultaneously.
- **The multiple partitioning schemes can be of two types:**
- Fixed Partitioning
- Dynamic Partitioning

Memory Management Techniques

- **Fixed Partitioning**
- The main memory is divided into several fixed-sized partitions in a fixed partition memory management scheme or static partitioning. These partitions can be of the same size or different sizes. **Each partition can hold a single process. The number of partitions determines the degree of multiprogramming, i.e., the maximum number of processes in memory.** These partitions are made at the time of system generation and remain fixed after that.
- **Advantages of Fixed Partitioning memory management schemes:**
- Simple to implement.
- Easy to manage and design.
- **Disadvantages of Fixed Partitioning memory management schemes:**
- This scheme suffers from internal fragmentation.
- The number of partitions is specified at the time of system generation.

Memory Management Techniques

- **Dynamic Partitioning**
- The dynamic partitioning was **designed to overcome the problems of a fixed partitioning scheme**. In a dynamic partitioning scheme, **each process occupies only as much memory as they require when loaded for processing. Requested processes are allocated memory until the entire physical memory is exhausted or the remaining space is insufficient to hold the requesting process**. In this scheme the **partitions used are of variable size**, and the **number of partitions is not defined at the system generation time.**
- **Advantages of Dynamic Partitioning memory management schemes:**
- Simple to implement.
- Easy to manage and design.
- **Disadvantages of Dynamic Partitioning memory management schemes:**
- This scheme also suffers from internal fragmentation.
- The number of partitions is specified at the time of system segmentation.

Memory Management Techniques

- **Dynamic Partitioning**
- The dynamic partitioning was **designed to overcome the problems of a fixed partitioning scheme**. In a dynamic partitioning scheme, **each process occupies only as much memory as they require when loaded for processing. Requested processes are allocated memory until the entire physical memory is exhausted or the remaining space is insufficient to hold the requesting process**. In this scheme the **partitions used are of variable size**, and the **number of partitions is not defined at the system generation time.**
- **Advantages of Dynamic Partitioning memory management schemes:**
- Simple to implement.
- Easy to manage and design.
- **Disadvantages of Dynamic Partitioning memory management schemes:**
- This scheme also suffers from internal fragmentation.
- The number of partitions is specified at the time of system segmentation.

Memory Management Techniques

- **What is paging?**
- Paging is a technique that eliminates the requirements of contiguous allocation of main memory. In this, the main memory is divided into fixed-size blocks of physical memory called frames. The size of a frame should be kept the same as that of a page to maximize the main memory and avoid external fragmentation.
- **In computer operating systems, memory paging is a memory management scheme by which a computer stores and retrieves data from secondary storage for use in main memory. In this scheme, the operating system retrieves data from secondary storage in same-size blocks called pages.**
- he main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames.
- One page of the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes.
- Pages of the process are brought into the main memory only when they are required otherwise they reside in the secondary storage.

Memory Management Techniques-Paging

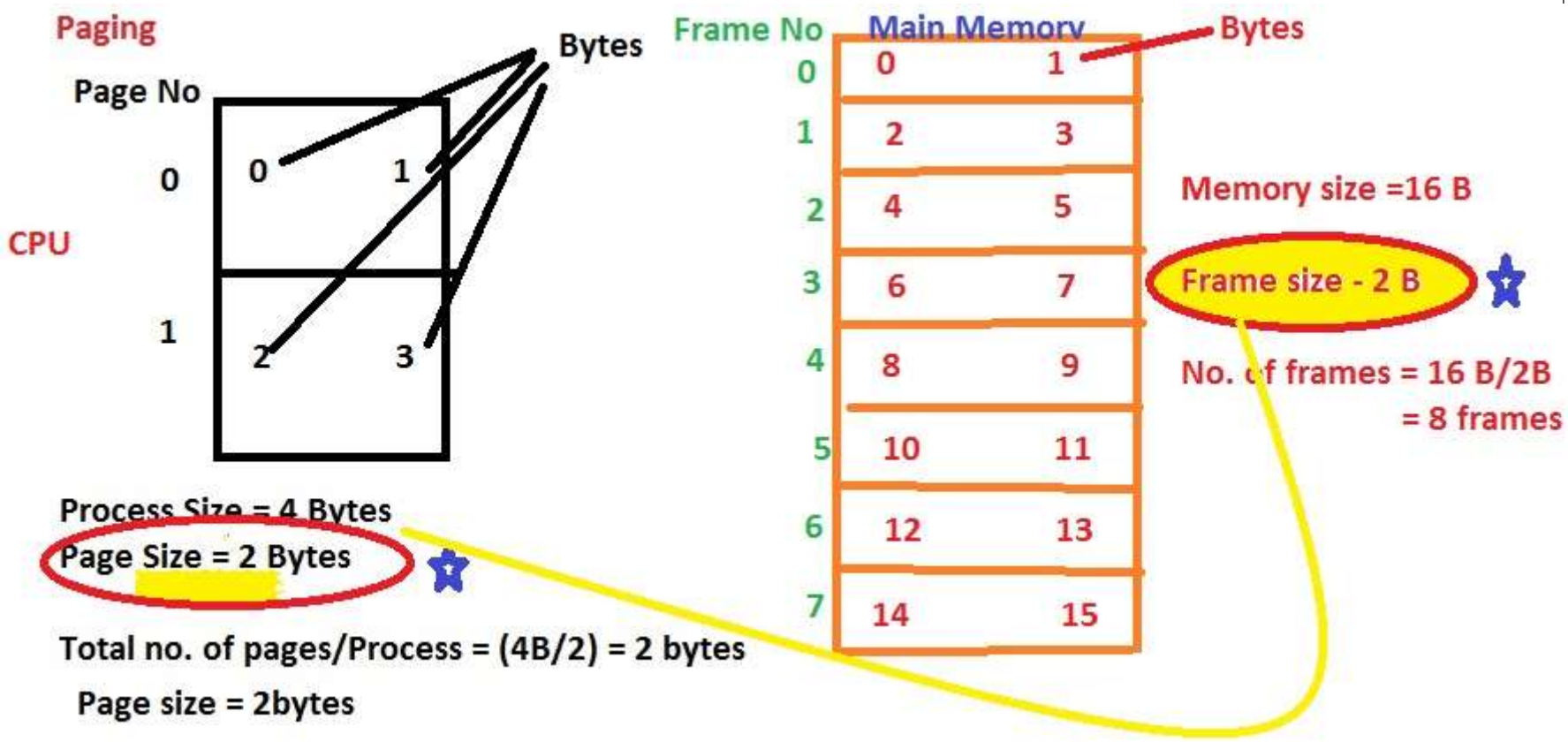
- **What is paging?**

- **Paged Memory Management**
- This method divides the computer's main memory into fixed-size units known as page frames. This hardware memory management unit maps pages into frames which should be allocated on a page basis.

- Different operating system defines different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.

- The pages can be stored at **the different locations of the memory** but the priority is always to find the contiguous frames or holes. Pages of the process are brought into the main memory only when they are required otherwise they reside in the secondary storage. Different operating system defines different frame sizes.

Memory Management Techniques-Techniques-Paging

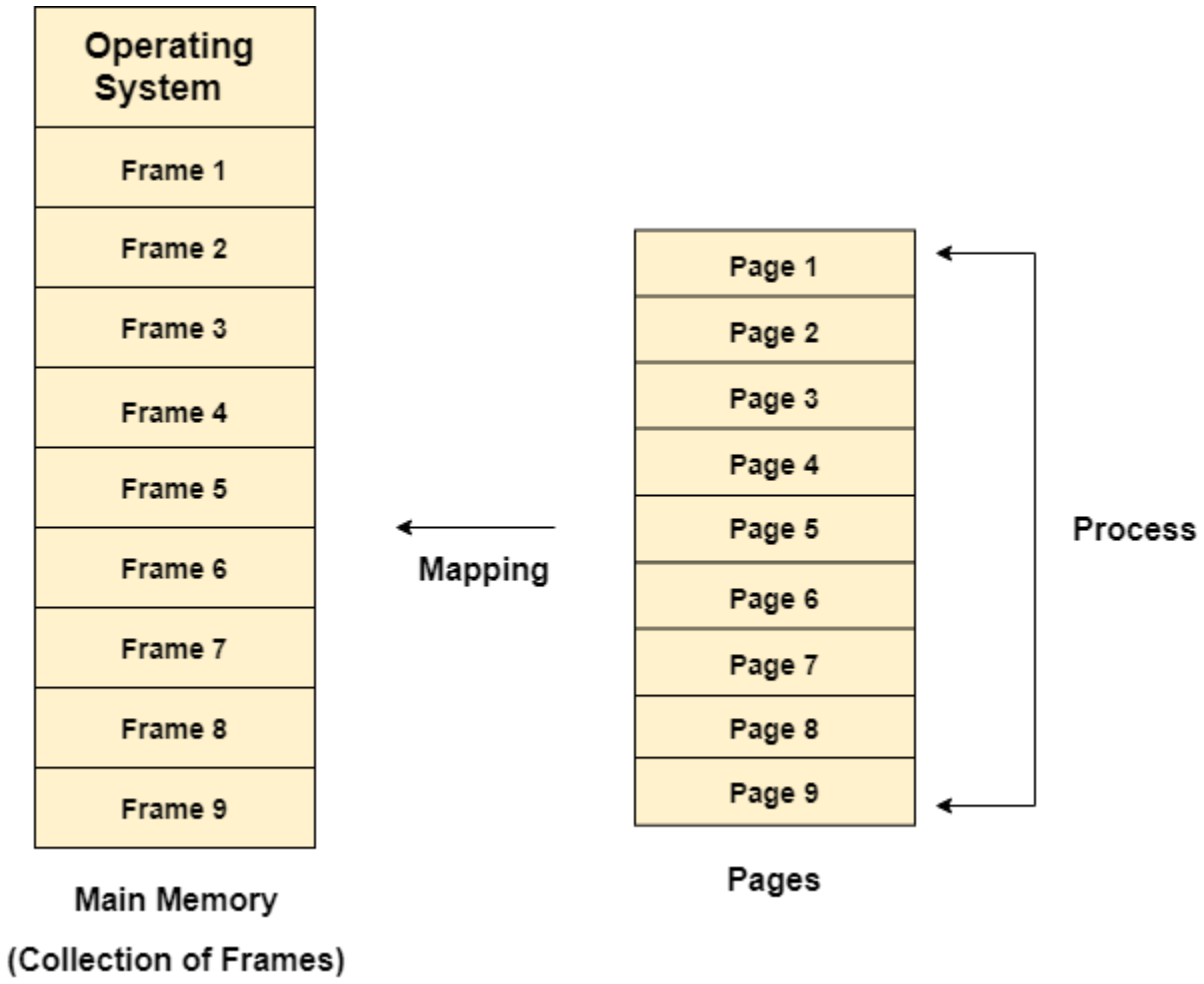


Memory Management Techniques-Techniques-Paging

- **Mapping with Page table**

Page Table of P1	
Page No.	Frame no.
O	F2
1	F4

Memory Management Techniques-Paging



Memory Management Techniques-Paging

Example

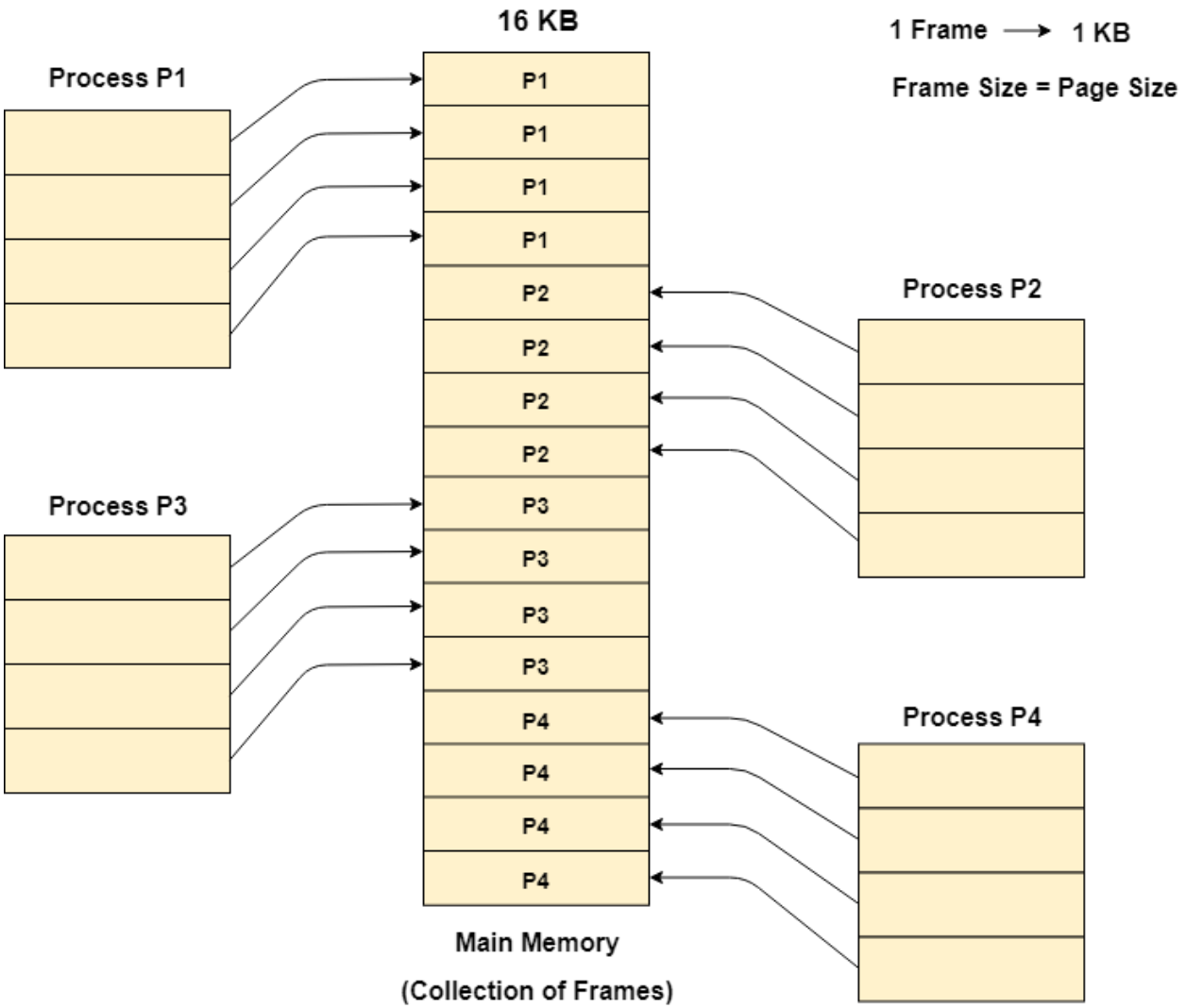
Let us consider the main memory size 16 Kb and Frame size is 1 KB therefore the main memory will be divided into the collection of 16 frames of 1 KB each.

There are 4 processes in the system that is P1, P2, P3 and P4 of 4 KB each. Each process is divided into pages of 1 KB each so that one page can be stored in one frame.

Initially, all the frames are empty therefore pages of the processes will get stored in the contiguous way.

Frames, pages and the mapping between the two is shown in the image below.

Memory Management Techniques-Paging



Paging

Memory Management Techniques

- **Advantages of paging:**
- Pages reduce external fragmentation.
- Simple to implement.
- Memory efficient.
- Due to the equal size of frames, swapping becomes very easy.
- It is used for faster access of data.

References

Links

- Reference
- https://www.youtube.com/watch?v=ITc09gOrqZk
- https://www.javatpoint.com/memory-management-operating-system
- https://www.guru99.com/os-memory-management.html
- https://www.youtube.com/watch?v=oqlXBn23fRI

Research Papers

Books

- 1. " Operating Systems: Internals and Design Principles" by William Stallings.
- 2. " Operating Systems: A Concept-Based Approach" by D M Dhamdhere.
- 3. System Concepts, 9th Edition, John Wiley & Sons, Inc. by Avi Silberschatz, Peter Baer Galvin, Greg Gagne,
- 4. D.M Dhamdhere: Operating systems - A concept based Approach, 3rd Edition, Tata McGraw-Hill, 2012.
- 5. Operating Systems: Internals and Design Principles, 8th edition Pearson Education Limited, 2014 by William Stallings.
- 6. Modern Operating system by Andrew Tenenbaum.
- 7. Distributed Operating System by Andrew Tanenbaum
- 8. P.C.P. Bhatt: Introduction to Operating Systems Concepts and Practice, 3rd Edition, PHI, 2010.
- 9. Harvey M Deital: Operating systems, 3rd Edition, Pearson Education, 2011